CS 229, Fall 2018

Problem Set #2: Supervised Learning II

Due Wednesday, Oct 31 at 11:59 pm on Gradescope

Notes: These questions require thought, but do not require long answers. Please be as concise as possible. If you have a question about this homework, please post it on the Piazza forum. You may not use any libraries except those included in the `environment.yml` file. Make sure your code runs without errors when executing `p05_percept.py` and `p06_spam.py`.:contentReference[oaicite:0]index=0

1 [15 points] Logistic Regression: Training Stability

We provide a logistic regression implementation in `src/p01_lr.py`, and two labeled datasets A and B in `data/ds1_a.txt` and `data/ds1_b.txt`.

Please do not modify the training algorithm code for this problem. Run logistic regression on both datasets by executing `python p01_lr.py` in the `src` directory.

(a) [2 points] What is the most notable difference in training the logistic regression model on datasets A and B?

At first glance, the main diference between both datasets is that the dataset B seems linearly separable while the dataset A is not.

(b) [5 points] Investigate why the training procedure behaves unexpectedly on dataset B, but not on A. Provide mathematical and/or experimental evidence to justify your claim. Explain why your argument does not apply to A. (Hint: It is not due to underflow/overflow.)

We see that the algorithm converges within 30300 iterations for the dataset A but it does not for the dataset B.

This is because the logistic regression cannot converge for a perfectly separable dataset. The idea behind this is the following.

The sigmoid function has the following limits

$$\lim_{\theta\to\infty} \sigma_\theta(x) = 0, \text{ if } x < 0$$

$$\lim_{\theta\to\infty} \sigma_\theta(x) = 1, \text{ if } x > 0$$

Lets suppose that our data is centered (therefore the intercept is zero) and the vertical line $x = 0$ can seperate our two classes. The function to optimize is the log-likelihood:

$$l(\theta) = \sum_i y_i \log \sigma_\theta(x_i) + (1 - y_i) \log (1 - \sigma_\theta(x_i))$$

This summation has two terms. The terms in which $y_i = 0$, look like $\log(1 - \sigma_\theta(x_i))$, and because of the perfect separation we know that for this terms $x_i < 0$. By the first limit we calculated, this means that

$$\lim_{\theta \to \infty} \sigma_\theta(x) = 0$$

for every x_i associated with a $y_i = 0$. Them, after appliying the log, we get that the limit is

$$\lim_{\theta \to \infty} \log(1 - \sigma_\theta(x_i)) = 0$$

The same ideas can be use for the terms with $y_i = 1$, obtaining:

$$\lim_{\theta \to \infty} \log(\sigma_\theta(x_i)) = 0$$

So no matter what θ is, you can always drive the objective function upwards by increasing θ towards infinity.

In the case on non perfectly separable points we would have points $y_i = 1$ associated with $x_i < 0$, and therefore the limit

$$\lim_{\theta \to \infty} \log(\sigma_\theta(x_i)) = -\infty$$

We would have also points $y_i = 0$ associated with $x_i < 0$, and therefore the limit

$$\lim_{\theta \to \infty} \log(1 - \sigma_\theta(x_i)) = 0.$$

Thus in between (because the log likelihood is continuous) there is a maximum of the function. Basically, this means that if there is even a single point that cannot be classified correctly, then making the parameters grow arbitrarily large drives the log-likelihood to minus infinity. Therefore, the algorithm cannot keep increasing the parameter norm, and a finite maximum of the log-likelihood must exist.

(c) [5 points] For each possible modification below, state whether it would cause the training algorithm to converge on datasets like B. Justify your answers.

i. **Using a different constant learning rate.**
This would not cause the algorithm to converge. This is because the problem is not the optimizer but the fact that the log-likelihood has no upper bound. Therefore the gradient never becomes zero and the parameters become larger without limit.

ii. **Decreasing the learning rate over time (e.g. scaling by $1/t^2$).**
This could case the gradient descent to stop but not because the algorithm has converged to a maximum (it cannot by definition). The parameters simply stop changing because the decreasing learning rate forces the updates to almost zero.

iii. **Linear scaling of the input features.**
A linear scale in the input features is the same as a linear scale in θ. This would not change the log-likelihood to be upper bounded.

iv. **Adding an ℓ_2 regularization term $\|\theta\|_2^2$.**
If we add a regularization term, the new optimization problem would be

$$l(\theta) = \sum_i \log(\sigma(y_i \theta^T x_i)) - \lambda ||\theta||_2^2.$$

If we suppose a perfectly separable problem then we would have $y_i\theta^T x_i > 0\ \forall i$. If now we make the limit

$$\lim_{\theta\to\infty} l(\theta) = \sum_i \log(\sigma(y_i\theta^T x_i)) - \lambda||\theta||_2^2 = -\infty$$

Now is not optimal to make θ larger and larger and therefore there must be a finite value of θ that makes our function maximum.

v. **Adding zero-mean Gaussian noise to the training data or labels.**
In this case the algorithm would converge because now the problem could be non perfectly separable. If the problem is not perfectly separable there exist a finite value of θ that makes the log-likelood maximum.

(d) [3 points] Are hinge-loss SVMs vulnerable to datasets like B under similar SGD procedures? Give an informal justification.

(Hint: geometric vs functional margin.)

By definition, the optimization problem of SVM is,

$$\min_\omega \frac{1}{2}||\omega||^2,\ \text{s.t}\ y_i(\omega^T x_i + b) \geq 1$$

This optimization can only be done if the data is perfectly separable because if not the conditions cannot be fulfilled. In the case of using a Hinge regularization, the Hinge loss will be minimized to zero and the algorithm would converge. Mathematically, Hinge loss is given by,

$$J(\hat{y}) = \max(0, 1 - y\cdot\hat{y}),\ \text{where}\ y = \omega^T x + b$$

If the data is linearly separable, then $y\cdot\hat{y} > 0$. Thus, if we increase ω abd b to make $|\hat{y}| > 1$, then $J(\hat{y}) = 0$

2 [10 points] Model Calibration

In this problem we explore why the logistic regression hypothesis $h_\theta(x)$ can be interpreted as a probability. We say a classifier is *well calibrated* if predicted probabilities match empirical frequencies.

Suppose we have training examples $\{(x^{(i)}, y^{(i)})\}_{i=1}^m$ with $x^{(i)} \in \mathbb{R}^{n+1}$, $y^{(i)} \in \{0,1\}$, and $x_0^{(i)} = 1$ for all i. Let θ be the maximum-likelihood estimate obtained by logistic regression. Define $I_{a,b} = \{i : h_\theta(x^{(i)}) \in (a,b)\}$ for $0 \leq a < b \leq 1$. A perfectly calibrated model must satisfy:

$$\frac{\sum_{i\in I_{a,b}} P\Big(y^{(i)} = 1 \mid x^{(i)}; \theta\Big)}{|I_{a,b}|} = \frac{\sum_{i\in I_{a,b}} \mathbb{1}\{y^{(i)} = 1\}}{|I_{a,b}|}.$$

(a) [5 points] Show that logistic regression satisfies the above calibration property over the probability range $(a,b) = (0,1)$ on the training data. (Hint: Use the bias term.)

The image of the sigmoid function is $Im(\sigma) = (0,1)$ and therefore the set $I_{(0,1)} = \{1, 2, ..., m\}$. So, the equality given in the header,

$$\frac{1}{m}\sum_i^m P(y^{(1)} = 1|x^{(i)};\theta) = \frac{1}{m}\sum_{i=1}^m \mathbb{1}\{y^{(i)} = 1\}$$

which is equal to,

$$\sum_{i}^{m} h_\theta(x^{(i)}) = \sum_{i=1}^{m} \mathbb{1}\{y^{(i)} = 1\} = \sum_{i=1}^{m} y^{(i)}$$

and the last equality holds because $y^{(i)} = \{0, 1\}$.

Now, if we recall the expression of the log-likelihood,

$$l(\theta) = \sum_{i} y_i \log \sigma_\theta(x_i) + (1 - y_i) \log (1 - \sigma_\theta(x_i))$$

and obtain the bias parameter that maximizes the log-likelihood

$$\frac{\partial l(\theta)}{\partial \theta_0} = \sum_{i} (y^{(i)} - h_\theta(x^{(i)})) x_0^{(i)} = \sum_{i} \left(y^{(i)} - h_\theta(x^{(i)}) \right) = 0$$

Thus,

$$\sum_{i} y^{(i)} = \sum_{i} h_\theta(x^{(i)}),$$

which the equality we wanted to show.

(b) [3 points] If a model is perfectly calibrated over all ranges $(a, b) \subseteq [0, 1]$, does it guarantee perfect classification accuracy? Is the converse true? Justify your answers. The answer of both questions is no. A model with perfect accuracy means that,

$$0.5 < P(y^{(1)} = 1 | x^{(i)}; \theta) < 1$$

Therefore, we would have,

$$\sum_{i \in I_{(0.5,1)}} P(y^{(1)} = 1 | x^{(i)}; \theta) < \sum_{i \in I_{(0.5,1)}} \mathbb{1}\{y^{(i)} = 1\}$$

because the left side is a sum of terms between 0.5 and 1, and the right side is a sum of terms equal to 1.

(c) [2 points] How does including ℓ_2 regularization in logistic regression affect model calibration? Discuss briefly.

In this case the log-likelihood is given by,

$$l(\theta) = \sum_{i} y_i \log \sigma_\theta(x_i) + (1 - y_i) \log (1 - \sigma_\theta(x_i)) - \frac{1}{2} \lambda ||\theta||_2^2.$$

In this case, the bias term that maximizes the log-likelihood is,

$$\frac{\partial l(\theta)}{\partial \theta_0} = \sum_{i} \left(y^{(i)} - h_\theta(x^{(i)}) \right) - \lambda \theta_0 = 0.$$

Thus,

$$\sum_{i} y^{(i)} = \sum_{i} h_\theta(x^{(i)}) + \lambda \theta_0.$$

This means that,

$$\sum_{i} y^{(i)} = \sum_{i} P(y^{(i)} = 1 | x^{(i)}; \theta) + \lambda \theta_0$$

and the model will not be well-calibrated.

3 [20 points] Bayesian Interpretation of Regularization

In Bayesian statistics, parameters such as $\boldsymbol{\theta}$ are treated as random variables with prior distributions. The posterior distribution $\boldsymbol{p(\theta \mid x, y)}$ encodes what we believe about $\boldsymbol{\theta}$ after observing data.

Rather than maximizing only the likelihood (MLE), the *maximum a posteriori* estimate (MAP) chooses $\boldsymbol{\theta}$ that maximizes the posterior:

$$\boldsymbol{\theta_{\text{MAP}} = \arg\max_{\theta}\ p(\theta \mid x, y).}$$

(a) [3 points] Show that

$$\boldsymbol{\theta_{\text{MAP}} = \arg\max_{\theta}\ p(y \mid x, \theta)\, p(\theta)}$$

under the assumption that $\boldsymbol{p(\theta) = p(\theta \mid x)}$, as is valid in models such as linear regression where $\boldsymbol{x}$ is not explicitly parameterized by $\boldsymbol{\theta}$.

The conditional probability of two events X and Y is given by,

$$\boldsymbol{p(Y|X) = \frac{p(Y, X)}{p(X)}}$$

and conditioned again on $\boldsymbol{\theta}$,

$$\boldsymbol{(Y|X, \theta) = \frac{p(X, Y|\theta)}{p(X|\theta)}}$$

We start from the conditional probability $p(\theta|x, y)$,

$$p(\theta|x, y) = \frac{p(\theta, x, y)}{p(x, y)}.$$

Using now the Bayes Theorem,

$$p(y|\theta, x) = \frac{p(\theta, x|y)p(y)}{p(\theta, x)},$$

$$p(y|\theta, x)p(\theta, x) = p(\theta, x|y)p(y),$$

And according to the definition of conditional probability,

$$p(\theta, x|y)p(y) = \frac{p(\theta, x, y)}{p(y)}p(y)$$

Thus,

$$p(\theta|x, y) = \frac{p(\theta, x|y)p(y)}{p(x, y)} = \frac{p(y|\theta, x)p(\theta, x)}{p(x, y)} = \frac{p(y|\theta, x)p(\theta)}{p(x, y)}$$

Finally, because we are maximizing the expression with respect to θ,

$$\arg\max_{\theta} p(\theta|x, y) = \arg\max_{\theta} \frac{p(y|\theta, x)p(\theta)}{p(x, y)} = \arg\max_{\theta} p(y|\theta, x)p(\theta) = \theta_{\text{MAP}}$$

(b) [5 points] Assume a Gaussian prior $\theta \sim \mathcal{N}(0, \eta^2 I)$. Show that MAP estimation is equivalent to MLE with ℓ_2 regularization:

$$\boldsymbol{\theta_{\text{MAP}} = \arg\min_{\theta} \Big[-\log p(y \mid x, \theta) + \lambda \|\theta\|_2^2 \Big].}$$

What is λ in terms of η?

If we start from the previous derivation (because $p(\theta|x) = p(\theta)$) and apply the log (because the log is a non-decreasing function is the same maximizing the $p(\theta|x, y)$ than $\log p(\theta|x, y)$), thus,

$$\theta_{\text{MAP}} = \arg\max_{\theta} p(y|\theta, x)p(\theta),$$

$$\log \theta_{\text{MAP}} = \arg\max_{\theta} \log p(y|\theta, x) + \log p(\theta).$$

Taking into account that $p(\theta) = \frac{1}{\sqrt{2\pi}} |\eta^2 I|^{-1/2} \exp\left(-\frac{1}{2}\left(\theta^T (\eta^2 I)^{-1} \theta\right)\right)$, we have

$$\log \theta_{\text{MAP}} = \arg\max_{\theta} \log p(y|\theta, x) - \frac{1}{2\eta^2}\theta^T\theta = \arg\max_{\theta} \log p(y|\theta, x) - \frac{1}{2\eta^2}||\theta||_2^2.$$

Which is equivalent to MLE with regularization if $\lambda = \dfrac{1}{2\eta^2}$

(c) [7 points] Consider linear regression: $y = \theta^T x + \varepsilon$ where $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, and prior $\theta \sim \mathcal{N}(0, \eta^2 I)$. Let X be the design matrix with rows $x^{(i)T}$ and $\vec{y}$ the output vector. Derive a closed-form solution for θ_{MAP}.

From the last exercise, we have that

$$\log \theta_{\text{MAP}} = \arg\max_{\theta} \log p(y|\theta, x) - \frac{1}{2\eta^2}\theta^T\theta = \arg\max_{\theta} \log p(y|\theta, x) - \frac{1}{2\eta^2}||\theta||_2^2.$$

In this case, we are given that $y^{(i)}|x^{(i)}, \theta \sim \mathcal{N}(\theta^T x^{(i)}, \sigma^2)$ and therefore

$$\begin{aligned}
\log \theta_{\text{MAP}} &= \arg\max_{\theta} \sum_{i=1}^{m} \log \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) - \frac{1}{2\eta^2}\|\theta\|_2^2 \\
&= \arg\max_{\theta} \Big(-\log(\sqrt{2\pi}\sigma) - \frac{1}{2\sigma^2}\sum_{i=1}^{m}(y^{(i)} - \theta^T x^{(i)})^2 - \frac{1}{2\eta^2}\|\theta\|_2^2 \Big) \\
&= \arg\min_{\theta} \Big(\log(\sqrt{2\pi}\sigma) + \frac{1}{2\sigma^2}\sum_{i=1}^{m}(y^{(i)} - \theta^T x^{(i)})^2 + \frac{1}{2\eta^2}\|\theta\|_2^2 \Big) \\
&= \arg\min_{\theta} \frac{1}{2\sigma^2}\sum_{i=1}^{m}(y^{(i)} - \theta^T x^{(i)})^2 + \frac{1}{2\eta^2}\|\theta\|_2^2 \\
&= \arg\min_{\theta} \frac{1}{2\sigma^2}(y - X\theta)^T(y - X\theta) + \frac{1}{2\eta^2}\|\theta\|_2^2
\end{aligned}$$

Now we want to find the θ that minimizes the expression. For this, we need to derive the expression and set it equal to zero,

$$\nabla_\theta\Big(\frac{1}{2\sigma^2}[y^Ty - y^TX\theta - \theta^TX^Ty + \theta^TX^TX\theta] + \frac{1}{2\eta^2}\|\theta\|_2^2\Big) = \frac{1}{2\sigma^2}\nabla_\theta(-2y^TX\theta + \theta^TX^TX\theta) + \frac{1}{2\eta^2}\nabla_\theta(\theta^T\theta) = \frac{1}{\sigma^2}(-X^Ty + X^TX\theta) + \frac{\theta}{\eta^2} = 0.$$

Thus,

$$\theta = (X^TX + \frac{\sigma^2}{\eta^2}I)^{-1}X^Ty$$

(d) [5 points] Now assume the prior is Laplace:

$$\boldsymbol{f_L(z \mid \mu, b) = \frac{1}{2b}\exp\left(-\frac{|z-\mu|}{b}\right), \quad \theta \sim L(0, bI).}$$

For linear regression $y = x^T\theta + \varepsilon$, show that MAP estimation corresponds to minimizing

$$\boldsymbol{J(\theta) = \|X\theta - \vec{y}\|_2^2 + \gamma\|\theta\|_1.}$$

What is γ in terms of b and σ? From (b) we have that,

$$\log\theta_{\text{MAP}} = \arg\max_\theta \log p(y|\theta, x) + \log p(\theta).$$

In this case we have that $\theta \sim L(0, bI)$ $p(\theta) = \frac{1}{2}(bI)^{-1}\exp\left(-(bI)^{-1}|\theta|\right)$. Therefore,

$$\log\theta_{\text{MAP}} = \arg\max_\theta \log p(y|\theta, x) - \frac{1}{b}|\theta|.$$

And because $y \sim \mathcal{N}(\theta^Tx, \sigma^2)$, we have,

$$\begin{aligned}\log\theta_{\text{MAP}} &= \arg\max_\theta \sum_{i=1}^m \left(-\frac{1}{2\sigma^2}(y^{(i)} - \theta^Tx^{(i)})^2 - \frac{1}{b}|\theta|\right)\\ &= \arg\max_\theta \left(-\frac{1}{2\sigma^2}\|y - X\theta\|_2^2 - \frac{1}{b}|\theta|\right)\\ &= \arg\min_\theta \left(\frac{1}{2\sigma^2}\|y - X\theta\|_2^2 + \frac{1}{b}|\theta|\right)\\ &= \arg\min_\theta \left(\|y - X\theta\|_2^2 + \frac{2\sigma^2}{b}|\theta|\right)\end{aligned}$$

Remarks: **Linear regression with ℓ_2 regularization is called Ridge regression; with ℓ_1 regularization, Lasso regression. Gaussian and Laplace priors induce *shrinkage* toward zero, and Lasso tends to yield sparse solutions.**

4 [18 points] Constructing Kernels

We seek to determine whether certain functions $K(x, z)$ are valid kernels, i.e. correspond to inner products in some feature space by Mercer's theorem.

Let K_1, K_2 be kernels over $\mathbb{R}^n \times \mathbb{R}^n$, $a \in \mathbb{R}^+$, $f : \mathbb{R}^n \to \mathbb{R}$, $\phi : \mathbb{R}^n \to \mathbb{R}^d$, K_3 a kernel over $\mathbb{R}^d \times \mathbb{R}^d$, and $p(x)$ a polynomial with positive coefficients.

For each function, state whether it is necessarily a kernel and justify:

(a) [1 point] $K(x, z) = K_1(x, z) + K_2(x, z)$

We just need to verify if $K(x, z)$ is PSD,

$$z^T K z = z^T K_1 z + z^T K_2 z \geq 0$$

because K_1 and K_2 are both PSD. Therefore, K is a valid kernel.

(b) [1 point] $K(x, z) = K_1(x, z) - K_2(x, z)$

We just need to verify if $K(x, z)$ is PSD,

$$z^T K z = z^T K_1 z - z^T K_2 z.$$

In this case $z^T K z$ could be lower than zero if $z^T K_2 z \geq z^T K_1 z$ and therefore K is not necessarily a kernel.

(c) [1 point] $K(x, z) = aK_1(x, z)$

We just need to verify if $K(x, z)$ is PSD,

$$z^T K z = a z^T K_1 z \geq 0$$

because $a \geq 0$ and K_1 is PSD. Therefore K is necessarily a kernel.

(d) [1 point] $K(x, z) = -aK_1(x, z)$

In this case K can not be a kernel because $z^T K(x, z) z = -a z^T K_1(x, z) z \leq 0$.

(e) [5 points] $K(x, z) = K_1(x, z)K_2(x, z)$ (Hint: It is a kernel; prove it.)

In this case we need the definition of K_1 and K_2,

$$\begin{aligned}
z^T K z &= z^T K_1 K_2 z \\
&= \sum_{i,j} z_i K_{ij} z_j \\
&= \sum_{i,j} z_i \, K_1(x^{(i)}, x^{(j)}) \, K_2(x^{(i)}, x^{(j)}) \, z_j \\
&= \sum_{i,j} z_i \, (\phi_1(x^{(i)})^T \phi_1(x^{(j)})) (\phi_2(x^{(i)})^T \phi_2(x^{(j)})) z_j \\
&= \sum_{i,j,k,m} z_i \, \phi_{1k}(x^{(i)}) \, \phi_{1k}(x^{(j)}) \, \phi_{2m}(x^{(i)}) \, \phi_{2m}(x^{(j)}) \, z_j \\
&= \sum_{i,j,k,m} (z_i \, \phi_{1k}(x^{(i)}) \, \phi_{2m}(x^{(i)})) (z_j \, \phi_{1k}(x^{(j)}) \, \phi_{2m}(x^{(j)})) \\
&= \sum_{k,m} \left(\sum_i z_i \, \phi_{1k}(x^{(i)}) \, \phi_{2m}(x^{(i)}) \right)^2 \\
&\geq 0.
\end{aligned}$$

(f) [3 points] $\boldsymbol{K(x,z) = f(x)f(z)}$

In this case,

$$z^T K z = z^T f(x) f(z) z = \sum_{ij} z_i f(x^{(i)}) f(x^{(j)}) z_j = \sum_i (z_i f(x^{(i)}))^2 \geq 0$$

(g) [3 points] $\boldsymbol{K(x,z) = K_3(\phi(x), \phi(z))}$

By definition K_3 is a kernel and therefore K is also a kernel.

(h) [3 points] $\boldsymbol{K(x,z) = p(K_1(x,z))}$

We have that,

$$K = p(K_1) = \sum_{i=0}^{n} c_i K_1^i.$$

We know that the sum of PSD matrices give a PSD matrix. We also know that aK is PSD if $a \geq 0$ and K is PSD. Finally from (e) we know that the product of two kernels is a valid kernel and thus, $K = (K_1)^i$ is a valid kernel.

Therefore $K(x,z) = p(K_1(x,z))$ is a valid kernel.

5 [16 points] Kernelizing the Perceptron

We consider the perceptron prediction rule $h_\theta(x) = g(\theta^T x)$, with $g(z) = \text{sign}(z)$. We apply one-pass SGD updates:

$$\theta^{(i+1)} := \theta^{(i)} + \alpha \, (y^{(i+1)} - h_{\theta^{(i)}}(x^{(i+1)})) \, x^{(i+1)}, \quad \theta^{(0)} = \vec{0}.$$

We wish to extend this to a high-dimensional feature space ϕ implicitly using a kernel K.

(a) [9 points] Describe how to kernelize the perceptron without ever explicitly computing $\phi(x)$:

i. **Representation of $\theta^{(i)}$ when $\theta^{(i)}$ lives in feature space of ϕ.**
From the update rule we can see that each update of $\theta^{(i+1)}$ is a linear combination of $x^{(i+1)}$ (ex. $\theta^1 = \alpha\left(y^{(1)} - h_{\theta^{(0)}}(x^{(1)})\right)x^{(1)}$). Therefore, the parameter vector $\theta^{(i)}$ can be represented as,

$$\theta^{(i)} = \sum_j^i \beta_j \phi(x^{(j)}); \;\; \theta^{(0)} = \sum_j^0 \beta_j \phi(x^{(j)}) = \vec{0}$$

ii. **Efficient prediction of $h_{\theta^{(i)}}(x^{(i+1)})$.**
Given the previous representation of $\theta^{(i)}$, we have

$$\begin{aligned} h_{\theta^{(i)}}(\phi(x^{(i+1)})) &= \text{sign}(\theta^{(i)^T}\phi(x^{(i+1)})) \\ &= \text{sign}\left(\sum_{j=1}^{i} \beta_j \, \phi(x^{(j)})^T \phi(x^{(i+1)})\right) \\ &= \text{sign}\left(\sum_{j=1}^{i} \beta_j \, K(x^{(j)}, x^{(i+1)})\right) \end{aligned}$$

iii. **Efficient update rule using only kernel evaluations.**
From the previous results,

$$\theta^{(i+1)} = \sum_{j=1}^{i} \beta_j \phi(x^{(j)}) + \alpha\left(y^{(i+1)} - \text{sign}\left(\sum_j^{i+1} \beta_j K(x^{(j)}, x^{(i+1)})\right)\right)\phi(x^{(i+1)}).$$

Therefore,

$$\beta_{i+1} = \alpha\left(y^{(i+1)} - \text{sign}\left(\sum_j^{i+1} \beta_j K(x^{(j)}, x^{(i+1)})\right)\right)$$

(b) [5 points] Implement your kernelized perceptron in `src/p05_percept.py`.

(c) [2 points] Train perceptrons using dot-product and RBF kernels on `data/ds5_train.csv` and evaluate on `data/ds5_test.csv`. Which kernel performs poorly and why?

6 [22 points] Spam Classification

We build a spam detector for SMS text using Naive Bayes and SVMs. Dataset: `data/ds6_spam_train.tsv` / `data/ds6_spam_test.tsv`.

(a) [5 points] Complete text processing functions: `get_words`, `create_dictionary`, `transform_text` in `src/p06_spam.py`. Dictionary and sample matrix will be saved to output.

(b) [10 points] Implement multinomial Naive Bayes with Laplace smoothing in `fit_naive_bayes_model` and `predict_from_naive_bayes_model`. Save predictions to `output/p06_naive_bayes_predictions`. (Hint: use log-probabilities to avoid underflow.)

(c) [5 points] Compute top five most indicative words for spam using:

$$\log \frac{P(\text{token } i \mid y = 1)}{P(\text{token } i \mid y = 0)}.$$

Save result to `output/p06_top_indicative_words`.

(d) [2 points] Tune RBF SVM radius to maximize validation accuracy. Write best radius to `output/p06_optimal_radius`.